

Predicting Buy Decisions in Professional Counter-Strike 2

Aaron Jackson

DSCI 570: Introduction to Machine Learning

1. Introduction

In professional Counter-Strike 2, a team's round-by-round buy decision, whether to “eco”, “force-buy”, “half-buy”, or “full-buy”, is one of the most consequential decisions an in-game leader makes, made under time pressure with no benchmark beyond that person's own experience. Analysts frequently point to economic discipline as one of the gaps separating top-tier rosters from developing ones, but no tool lets a team check a specific decision against how tier-1 teams have handled the same situation. This project builds a machine learning tool that, given a team's own current game state (score, round win/loss streak, starting bankroll, side, etc.), recommends the buy-type decision most consistent with how professional teams have handled similar situations historically.

1.1. Background

Counter-Strike 2 (CS2) is a 5v5 tactical first-person shooter and one of the largest esports in the world. Teams play as either Counter-Terrorists (CT) or Terrorists (T), swapping sides after twelve rounds; the first team to win thirteen rounds wins the map. Each round opens with a twenty-second buy phase in which the team spends accumulated money on weapons, armor, and utility grenades.

Money carries across rounds, which is what makes the buy decision non-trivial. Teams earn income from round wins, losses, kills, and objectives, and any money left unspent remains available the following round. A team that spends everything on a marginal buy may still be likely to lose, and then be unable to arm itself for the next two rounds as a consequence. In contrast, a team that instead chooses to save up money and concede the current round in exchange for a fully-equipped one after it is oftentimes the smarter choice.

The loss bonus makes this tradeoff explicit. A team receives an escalating payment for each consecutive round lost: \$1,400 for the first, then \$1,900, \$2,400, \$2,900, and \$3,400 for the fifth and beyond, and this decreases for every round won. Losing deliberately is not a strategy, but knowing what a loss would pay is central to deciding how much to risk in the current round.

This produces the four buy types. An “eco” is a minimal spend, under roughly \$700 per player, deliberately saving for a later round. A “full-buy” is a complete loadout of rifle, armor, and utility, costing roughly \$4,400 per player on CT and \$3,900 on T, since CT weapons and utility are more expensive. Between these lie two decisions that can involve identical spending but mean different things: a “half-buy” leaves the team able to full-buy next round even if this one is lost, while a “force-buy” does not. The distinction is forward-looking rather than quantitative, which is what makes it difficult to evaluate under time pressure and what makes it a useful prediction target.

1.2. Audience and Motivation

The intended audience of this project are coaches and in-game leaders at any competitive tier, applying the tool for post-match review. The twenty-second buy phase leaves no room to consult a model mid-match, but the rounds after a match are exactly when a team asks whether a given call was defensible. For amateur and semi-professional rosters, the tool answers a question they otherwise have no way to check: whether a decision matched what a tier-1 team would have done from the same position. For professional teams, whose own decisions constitute the training data, the value is narrower but still potentially useful as it will still flag rounds where a call departed from established practice and could indicate a lapse in discipline.

As such, the motivation for this project is both personal and practical. The author follows professional Counter-Strike closely and also competes in the ESEA Main League, encountering this decision from both sides: watching professionals make it under scrutiny, and making it in-game with far less information. Buy decisions are among the few in Counter-Strike that are fully determined by observable state rather than mechanical skill, which makes them unusually well suited to modeling. A team cannot practice aim by consulting a dataset, but it can review whether its economic decisions matched what a tier-1 roster would have done in the same position.

1.3. Definition of Success

Raw accuracy is a poor evaluation metric for this problem. Full-buys account for roughly 70% of rounds in professional play, so a model that predicts "full-buy" unconditionally would report about 70% accuracy while providing no information. The majority-class baseline is therefore the floor any result must be measured against.

The decision that actually matters the most is whether to eco or force-buy. These are the situations where economic discipline separates rosters, and where a wrong call is costliest. A mistimed force-buy can cost a team both the current round and the two after it. Full-buys are already most teams' default guess; predicting it correctly is necessary but not the interesting part.

We will define this tool as useful if it correctly flags a genuine eco or force-buy situation at least 70% of the time it occurs. Below that threshold, a coach reviewing rounds would reasonably learn to ignore it. The secondary target was 80% overall four-class accuracy with macro-F1 comfortably above the majority-class baseline. Macro-F1 is the primary metric we will use throughout, because it weighs all four classes equally and therefore cannot be gamed by predicting the dominant class.

2. Technical Details

2.1. The Dataset

No public dataset exists in this format, so the dataset was constructed for this project. Match replays (.dem files) from three tier-1 tournaments (IEM Cologne 2026, BLAST Rivals 2026 Season 1, and IEM Rio 2026) were downloaded manually from HLTV.org, which hosts them free to the public. Files were extracted from their distributed archives and parsed with the open-source Python libraries Awpy and demoparser2.

One row is one team's state for one round, sampled at the start of freeze-time (the moment the buy phase starts, 20 seconds before the round begins). The target, in contrast, is produced using the tick at the very end of freeze-time (the moment a team's buy is locked in). Each round therefore produces two rows, one per team, and every column is written from that team's own perspective (`own_score` is the score of the team on that row, `opp_score` is their opponent's score). For instance, a team would never truly know the exact economic state of their opponents, so data about the opponent's bankroll is excluded.

As well, "pistol rounds" (rounds 1 and 13) are excluded from the output. Both teams begin those rounds with a fixed \$800 per player, so there is no decision to model: the state is identical every time and every team buys the same way. These rounds are still parsed internally however, because rounds 2 and 14 need to know some details of what happened immediately before them.

In total, 185 maps produced 7,030 team-round observations.

Group	Columns
Own economy	<code>cumsum_spare_money</code> , <code>cumsum_equip_value</code> , <code>max_spare_money</code> , <code>min_spare_money</code> , <code>num_players_gt_1000</code> , ..., <code>num_players_gt_6000</code>
Match state	<code>round_num</code> , <code>is_ot</code> , <code>is_ct</code> , <code>own_score</code> , <code>opp_score</code> , <code>own_loss_bonus</code> , <code>opp_loss_bonus</code> , <code>round_streak</code>
Previous round	<code>prev_round_buy_type</code> , <code>own_survivors_prev_round</code> , <code>opp_survivors_prev_round</code> , <code>awps_saved</code> , <code>rifles_saved</code> , <code>smgs_saved</code>

Table 1: Features (24 model inputs). Grouped by what they describe.

`cumsum_spare_money` is the team's total unspent money entering the buy phase; `cumsum_equip_value` is the value of equipment surviving players carried in from the previous round. The six `num_players_gt_N` columns count how many of the five players hold more than \$ N , which captures the distribution of money rather than just the total. Four players with \$3,000 each and one player with \$15,000 sum identically, but imply very

different buys. `round_streak` is signed relative to the team on the row: positive means they are on a winning streak, negative a losing one. Three columns in the dataset are deliberately not model inputs: `demo` is the grouping key for splitting by match (see 2.4), `team` is a roster identifier, and `buy_type` is the target.

The target is a four-class label (eco, half-buy, force-buy, full-buy) that is assigned by a rule evaluated at the last tick of freeze-time when the team's actual spending is known. The rule applies in order:

If carried equipment plus new spending meets the side-specific full-buy threshold (\$22,000 team total on CT, \$19,500 on T), label it a full-buy. Equipment already held counts, so a fully-kitted team that spends nothing after a clean win is correctly a full-buy rather than an eco. Otherwise, if new spending is at or below \$3,500 for the team, label eco. This check ignores carried equipment, so a team where one survivor kept a rifle while everyone else buys pistols still reads as an eco. Otherwise, project the worst case: assume the round is lost, step the loss bonus up one level, and check whether remaining money plus that bonus would cover a full-buy next round. If yes, then half-buy; if no, then force-buy. The only exception is on the last round before a side swap. Any mid-range buy is labeled a force-buy, since there is no next round on that side to protect. This is done deliberately because no professional team would ever eco or half-buy on these rounds.

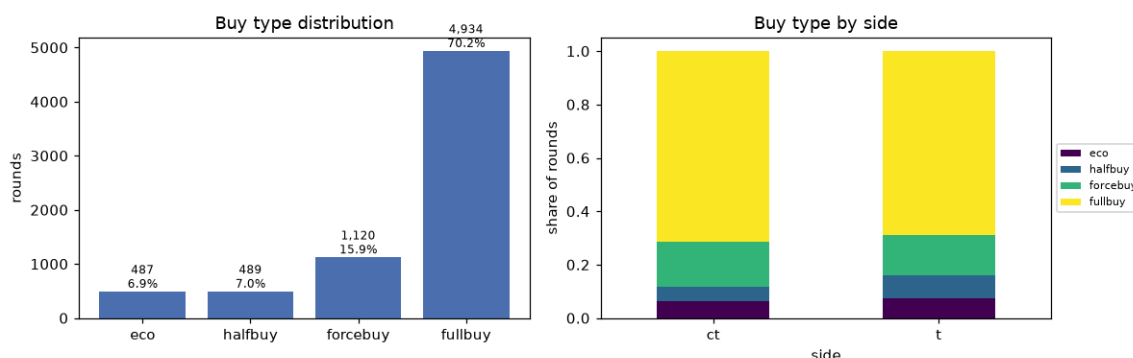


Figure 1: Class distribution across 7,030 observations. Full buys dominate at roughly 70%, which is why macro-F1 rather than accuracy is the primary metric.

It would be difficult to produce a fully-worked example of the data due to the number of features, so the following figure showcases a worked example using a subset of the features from a single observation. This observation is from the perspective of G2, playing against Astralis at Blast Rivals Season 1 on the first map of `de_dust2`.

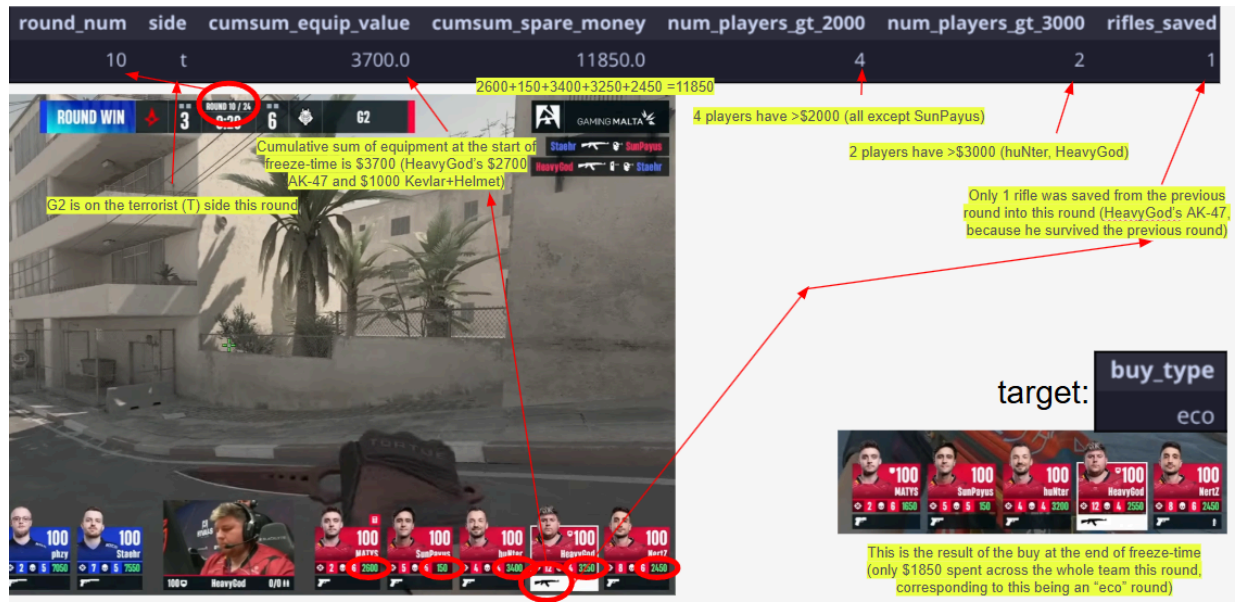


Figure 2: Annotated observation on a subset of features from G2 vs Astralis at Blast Rivals Season 1.

2.2. Data Wrangling and Cleaning

The parsing pipeline was written specifically to emit a model-ready dataset, so no post-hoc cleaning was required. The engineering effort went into the extraction itself, which involved four non-obvious problems.

The first was snapshot timing. Player state must be sampled at a specific tick. Testing confirmed that `start_balance` and `round_start_equip_value` are fixed for the entire round, so a single snapshot at the very start of freeze-time captures both the pre-buy economy. A simple second tick sample at the very end of freeze-time was used to capture the target. Snapshots are bounded within the round's own tick range so that pauses and timeouts elsewhere in the demo cannot leak in.

The next is the "phantom" pistol value. Default pistols are issued free at round start and can never be purchased, but the raw equipment-value statistic still prices them at \$200. Left uncorrected, every player appears to carry an extra \$200 of equipment they did not buy. The pipeline subtracts this phantom value per player unless that player survived the previous round holding a pistol they actually purchased.

Then there was determining half and overtime boundaries. These behave differently and require separate handling. At halftime, CS2 wipes all equipment and resets both teams' loss bonus, so momentum streak, loss bonus, and carried equipment all reset. Overtime halves swap sides and top both teams up to \$10,000, so the link to the previous round's buy had to be removed for the start of every overtime half.

Finally, there was the issue of team perspective. The parser tracks running state (score, loss bonus, momentum) keyed to a fixed roster identified in round 1. An early version of the pipeline emitted these as absolute `home_*` and `away_*` columns, meaning that on every away-team row the model was reading the opponent's score and an inverted momentum streak. This was corrected so that every emitted column is written from the perspective of the team on that row.

2.3. Model

Tree ensembles were selected for this problem: Random Forest and histogram-based gradient boosting, both offered by the scikit-learn library. The reasoning for this choice is that buy decisions are threshold-driven rather than smooth. A team below a certain bankroll would generally eco; above another it would generally full-buy. Decision trees partition on thresholds natively, so this structure is what they represent most directly. They also require no feature scaling, handle the mixed continuous-and-count feature space without preprocessing, and also remain inspectable and interpretable.

A neural network was considered and rejected. With roughly 7,000 observations and 24 tabular features, the data volume does not justify the capacity, and learned representations would forfeit the interpretability that makes the tool useful to a coach or in-game-leader.

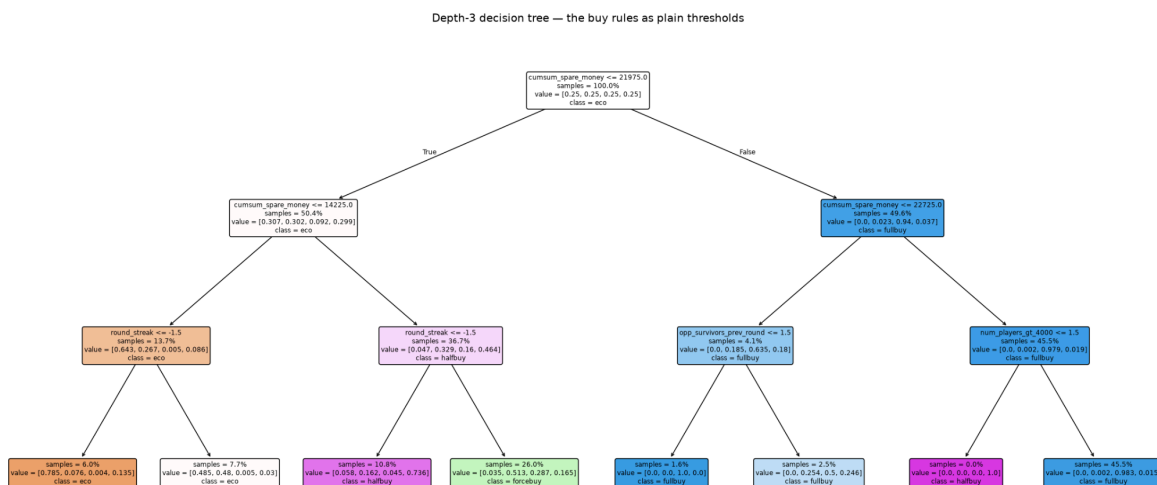


Figure 3: A deliberately shallow decision tree fit to the same data, showing the buy rules as explicit dollar thresholds. This is not the deployed model, but it illustrates the threshold structure the ensembles exploit.

2.4. Model Training

Feature learning was deliberately not used. Rather than allowing a model to discover its own representations from raw data, features were constructed directly from the CS2 rulebook: the loss-bonus schedule, side-specific full-buy costs, and the distribution of money across the five players. Feature learning requires substantially more data and produces

representations that cannot be easily interpreted. Hand-engineering requires domain knowledge, but produces columns with units a coach can reason about. With extensive domain knowledge available, only a few thousand observations, and interpretability as a product requirement, the second approach wins on all fronts.

Data was split by match, not by round. Whole demos are assigned entirely to training, validation, or test, in a (roughly) 64/16/20 split of matches, leaving 1,352 rounds across 37 held-out matches for final evaluation. This was done explicitly because one feature in the dataset is `prev_round_buy_type`, the buy type from the round immediately before. A round-level split could place round 8 in training and round 7 in test, where round 7's label is exposed as 8's feature. As a result, the model would score well on information it should not have. Beyond that specific leak, rounds within a match are also strongly autocorrelated: the same two teams, the same map, and an economy that compounds from round to round and is dependent entirely on previous rounds.

Hyperparameter tuning used randomized search over roughly 70 configurations with 5-fold StratifiedGroupKfold cross-validation, scored on macro-F1. The stratified-group variant keeps whole matches inside a single fold while ensuring the rare classes remain represented in every fold. In contrast, plain GroupKfold could produce folds containing almost no eco rounds. Searched parameters used in hyperparameter tuning included tree count, depth, minimum samples per leaf, features per split, and class weighting for the forest; iterations, learning rate, leaf count, and L2 regularization for gradient boosting.

The test set was not evaluated against until all modeling decisions were final.

2.5. Performance and Results

The tuned gradient boosting model achieved a cross-validated macro-F1 of 0.819 during the search, selecting a slow, shallow, heavily-regularized configuration (learning rate 0.03, 15 leaf nodes, 400 iterations, balanced class weights). That the search preferred less model complexity than the defaults is itself consistent with a decision space governed largely by thresholds on a handful of money features.

Metric	Model	Majority Baseline	vs. Baseline
Accuracy	91.1%	69.7%	+21.4 pts
Macro-F1	0.818	0.205	4.0x

Table 2: Held-out performance on 1,352 rounds from 37 unseen matches.

Class	Recall	Support	Meets 70% Bar
Full-Buy	96.5%	943	—
Force-Buy	81.8%	225	Yes
Half-Buy	75.5%	94	—
Eco	74.4%	90	Yes

Table 3: Per-class recall, ordered easiest to hardest. Full-buy and half-buy are not the product-critical classes, and so are labeled with a ‘—’ under the Meets 70% Bar column.

Both product-critical classes clear the 70% threshold with substantial margin, and overall accuracy exceeds the 80% target suggested in (1.3).

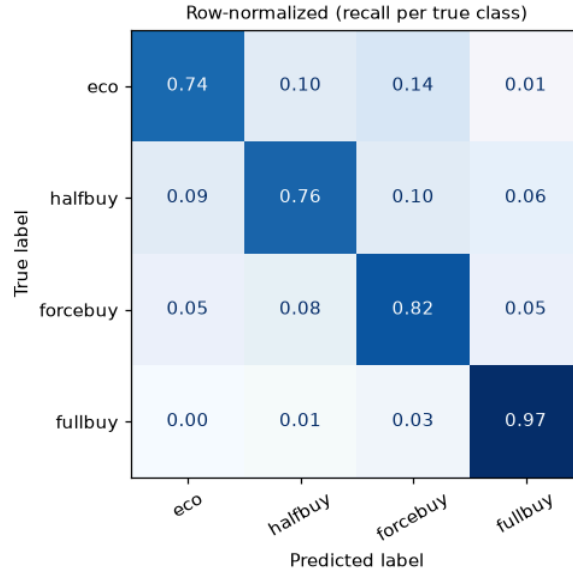


Figure 4: Confusion matrix on held-out matches. Errors concentrate adjacent to the diagonal.

68% of all misclassifications (81 of 120) fall between adjacent spend levels; only 39 span two or more. The model rarely mistakes an eco for a full-buy, it misses by just one step. This indicates it has learned the ordering of the four decisions rather than treating them as unrelated categories, which is a much more useful failure than being wrong at random.

True	Predicted	Rounds	Share of true class
Eco	Force-Buy	13	14%
Eco	Half-Buy	9	10%
Half-Buy	Force-Buy	9	10%
Half-Buy	Eco	8	9%

Table 4: The dominant confusions.

Eco was the weakest class, as nearly a quarter of genuine ecos are predicted as some form of spending. This reflects a structural limit of the feature set. The label is determined by what a team spends; the features describe what a team has. Eco is the only one of the four decisions that is purely voluntary, the other three are at least partly constrained by affordability. This means that a team sitting on \$12,000 that chooses to save is, on these features, indistinguishable from a team about to force with the same \$12,000.

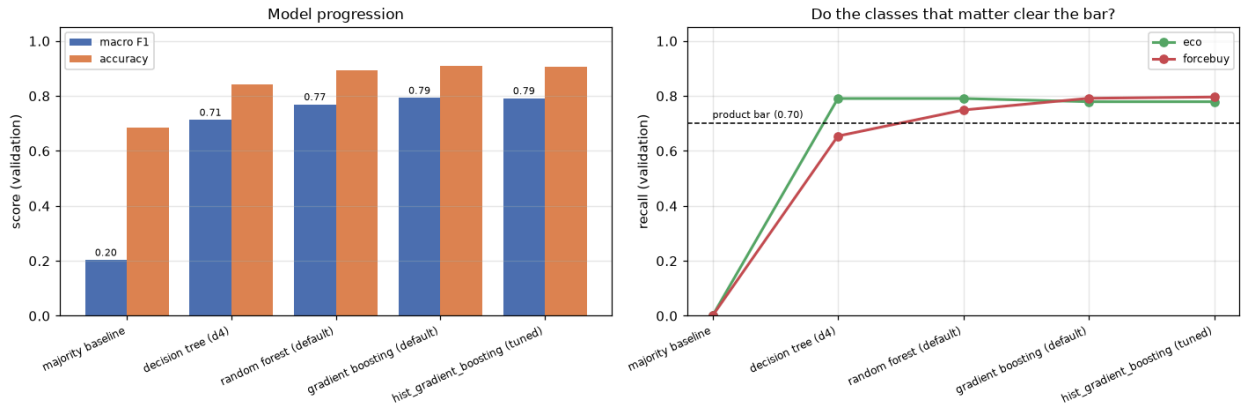


Figure 5: Macro-F1 and accuracy across development stages (left), and recall on the two product-critical classes against the 70% bar (right).

Stage	Macro-F1	Accuracy
Majority Baseline	0.203	68.3%
Decision Tree (depth 4)	0.713	84.3%
Random Forest (defaults)	0.769	89.3%
Tuned model	0.791	90.7%

Table 5: Development of macro-F1 and accuracy scores as the model progressed.

Note that Table 5 reports validation-set scores, since all five stages were fit on training data alone to keep the comparison like-for-like. Table 2 reports the final test-set result on matches untouched until modeling was complete, so the two tables do differ slightly.

The shape of this progression is more informative than its endpoint. Macro-F1 rises from 0.203 to 0.791 overall, but 87% of that gain arrives with the very first model, which is a rather simple decision tree limited to depth four. Every subsequent step, including a full randomized hyperparameter search, contributes the remaining 13%. This is consistent with the threshold-structure argument in (2.3), as if the decision boundary really is a small set of dollar cutoffs, then a shallow tree should capture most of it, and additional model capacity has little left to find. The tuning result reinforces this, since the search converged on a slower, shallower, more heavily regularized configuration than the default hyperparameters.

2.6. Interpretability

Permutation importance was computed on the held-out test set and scored by macro-F1. Each feature is then shuffled in turn, and the resulting drop in performance is measured. Because it is computed on unseen data, it avoids the bias toward high-cardinality features that affects impurity-based importance. It is worth mentioning that permutation importance indicates that a feature matters, not in which direction it pushes a prediction. The depth-3 tree in Figure 3 shows the actual thresholds (albeit on a simpler model).

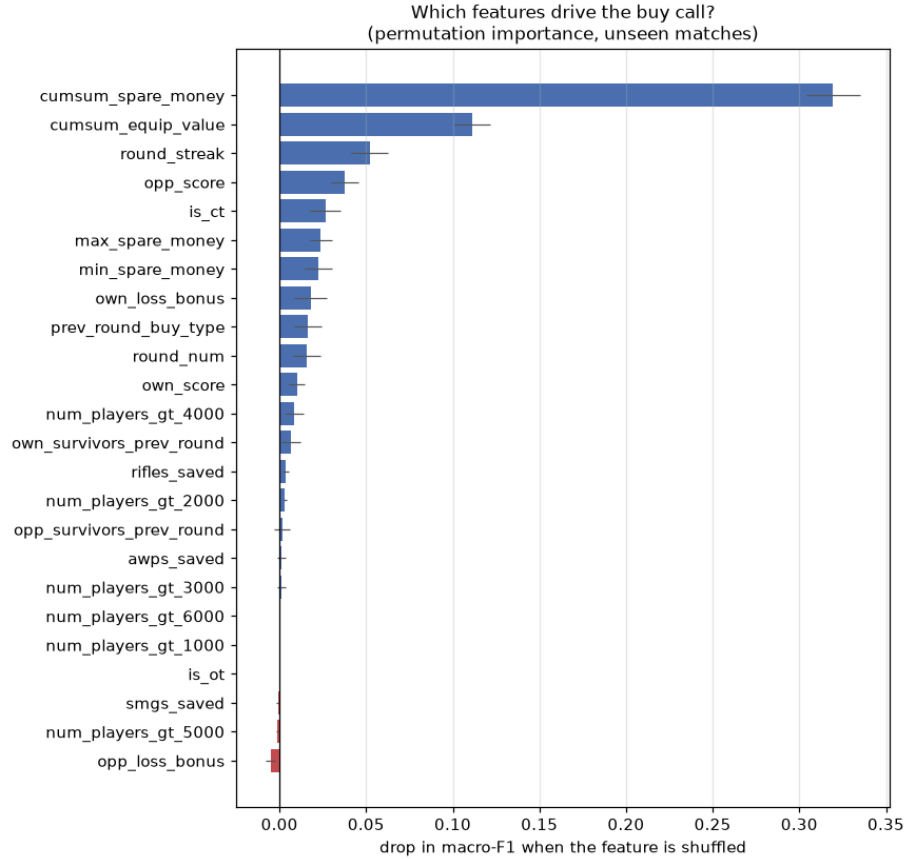


Figure 6: Permutation importance on held-out matches. Bars show mean drop in macro-F1 when the feature is shuffled.

Team bankroll and carried equipment dominate by a wide margin, confirming the hypothesis that economic state carries most of the signal. The three lower-ranked results were less obvious, and two of them are not directly economic.

`round_streak` ranked third, indicating that momentum carries signal that raw economy does not explain. Teams on a losing run will behave differently than bankroll alone predicts. `opp_score` ranked fourth. The plausible reason for this is match pressure: a team facing match point will always spend what it has rather than saving. Furthermore, the opponent's score also rises with the round number, so some of the signal may reflect match progression generally. `is_ct` ranks fifth, which follows mechanically from CT full-buys costing roughly \$500 more per player compared to T full-buys.

2.7. Real Application

As a practical test, the trained model was applied to a match from the author's own amateur team (playing in the ESEA Main League), a lower competitive tier than the tier-1 professional matches it was trained on.

The model agreed with the team's actual call on 81% of 21 rounds. This match was full-buy heavy, so an always-full-buy baseline would score 71% on it; the sample is small and the result should be read as indicative rather than conclusive.

The disagreements are more informative than the agreements. Three of the four are in the same situation: the team holds \$15,000–\$21,000 with little carried equipment and ecos, while the model predicts a force or half-buy at over 94% confidence. This reproduces the eco failure mode from the held-out test set on data from an entirely different skill tier, which is further evidence that the limitation may be structural rather than an artifact of the training distribution.

3. Challenges and Conclusion

3.1. Did It Work?

Yes, against the criteria set before development began. Eco recall reached 74.4% and force-buy recall 81.8%, both clearing the 70% threshold defined as the point below which a coach would stop consulting the tool. Overall accuracy of 91.1% exceeded the 80% target proposed, and macro-F1 of 0.818 is four times the majority-class baseline. Furthermore, the fact that 68% of errors fall between adjacent spend levels means the model learned the ordering of these decisions rather than four arbitrary labels.

3.2. Challenges

The hardest problems were all upstream of the model. Acquiring and parsing the demo files, and above all defining the labels, consumed far more effort than the model selection or tuning process.

Establishing team perspective was one such issue early on in the demo parsing process. The parser tracks state against a fixed roster, and the initial implementation produced absolute rather than team-relative columns. Every “away”-team row was therefore reading the opponent's score and an inverted momentum streak. This error was subtle, and went unnoticed for a while because the model still trained and still produced plausible numbers.

And as mentioned before, leakage was a significant issue in two distinct forms. The first was the opponent's bankroll, present in the demo file, invisible at freeze-time, and therefore unusable regardless of how much it helped. The second was the splitting strategy: with `prev_round_buy_type` as a feature, a round-level split would insert a training label directly into a test feature.

Lastly, data volume was also a considerable issue. Demo files average roughly 500-700 MB per match, which practically limited how many matches could be downloaded and processed.

3.3. What Should've Been Done Differently & Next Steps

There are a couple things that should've been done differently in retrospect. One of these would be to validate the labels against hand annotation, as this would provide the single highest-leverage change. The labeling rule was spot-checked against a handful of matches, but all 7,030 observations were never independently verified. Ideally, if it was possible to obtain a hand-annotated sample of all rounds would be fantastic, though unrealistic. However, manually annotating a couple hundred rounds would establish how closely the heuristic tracks human judgment, and every result in this report is downstream of that rule being correct. The second would be to collect more matches. This would by far be the

cheapest available improvement, particularly for the minority classes of eco and half-buy which have fewer than 100 test examples each, which makes their recall estimates more susceptible to noise than is ideal.

3.4. What Was Learned

The labeling methodology mattered more than the model choice. Utilizing the two model families and roughly 70 hyperparameter configurations produced results within a rather narrow band of each other. However, decisions about what counts as a force-buy or not determined the entire shape of the problem. Every number in this report is downstream of a rule written by hand and spot-checked over a handful of observations.

Second, leakage is extremely easy to introduce accidentally and difficult to detect from performance metrics, especially since a leaking model looks better, not worse. Both leaks in this project were found by asking what information a user would actually have in any given round state and carefully checking how the data is passed through the pipeline, not by observing anything wrong in the results.

4. Acknowledgements and References

Parsing and data extraction

- Awpy 2.0.2 — <https://github.com/pnxenopoulos/awpy>
- demoparser2 — <https://github.com/LaihoE/demoparser>

Modeling and analysis

- scikit-learn, pandas, NumPy, matplotlib

Data

- Professional match demo files from HLTV.org and the respective tournament organizers (ESL, BLAST)

CS2 economy values

- Author's domain knowledge, verified in-game and against parsed demo data